Bases de Datos a Gran Escala

Sesión 6 · Bases de datos de grafos: Neo4j

Diego Sevilla Ruiz
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Introducción a las bases de datos de grafos

Introducción a las BBDD de grafos

- Basadas en el concepto matemático de grafo
- **Nodos** o **vértices**, que contienen un identificador y un conjunto de propiedades
- Un conjunto de **arcos** o **aristas** que conectan pares de nodos, de forma dirigida o no; los arcos pueden tener sus propios atributos
- En el caso particular de Neo4j, los nodos también pueden tener un conjunto de etiquetas (***label***) y pares ⟨clave, valor⟩. La etiqueta equivale a su «tipo»
- Incluyen un lenguaje de búsqueda de elementos y manipulación del grafo

Ventajas de las BBDD de grafos

- **Navegación eficiente**: recorrido de relaciones en tiempo constante mediante *index-free adjacency*
- **Flexibilidad de esquema**: fácil añadir nuevos tipos de nodos y relaciones sin afectar a los datos existentes
- **Consultas intuitivas**: el lenguaje refleja la estructura visual del grafo (*ASCII art*)
- **Modelado natural**: muchos dominios son inherentemente grafos (redes sociales, dependencias, rutas)
- **Análisis de relaciones**: algoritmos especializados para detectar patrones, comunidades, rutas óptimas

Desventajas y limitaciones

- **Operaciones agregadas masivas**: menos eficientes que los RDBMS para operaciones OLAP
- **Escalado horizontal limitado**: Neo4j Community sólo soporta una instancia; Enterprise usa replicación *master-slave*
- **Consultas globales costosas**: recorrer todo el grafo puede ser ineficiente
- **Curva de aprendizaje**: requiere pensar de forma diferente al modelo relacional
- **Tamaño de datos**: mejor rendimiento cuando el grafo completo cabe en memoria

Casos de uso principales

- **Sistemas de recomendación**: productos, contenidos, amigos (Amazon, Netflix, LinkedIn)
- **Detección de fraude**: patrones sospechosos en transacciones financieras (PayPal, eBay)
- **Análisis de redes sociales**: influencia, comunidades, propagación de información
- **Gestión de identidad y acceso**: permisos, roles, jerarquías organizacionales
- **Análisis de dependencias**: software, cadenas de suministro, infraestructura IT
- **Gestión del conocimiento**: *knowledge graphs*, ontologías, grafos de conceptos
- **Redes de transporte**: rutas óptimas, análisis de tráfico

Comparación con otros paradigmas

Relaciones	`JOIN`s costosos	Referencias limitadas	Nativas y eficientes
Esquema	Rígido	Flexible	Muy flexible
Consultas	SQL	*Query DSL*	Cypher/Gremlin
Mejor para	OLAP, agregaciones	Documentos aislados	Datos conectados
Escalado	Vertical/*sharding*	Horizontal	Limitado (Neo4j)

Elegir grafos cuando: las relaciones son tan importantes como los datos

Property Graph vs. RDF

Dos modelos principales de bases de datos de grafos:

- **Property Graph** (Neo4j, Neptune, CosmosDB): nodos y relaciones con propiedades, relaciones con nombre y dirección, lenguajes Cypher/Gremlin — orientado a aplicaciones
- **RDF** (*Resource Description Framework*): tripletas sujeto-predicado-objeto, ontologías y razonamiento lógico, lenguaje SPARQL, bases de datos Blazegraph/GraphDB/Stardog — orientado a web semántica y conocimiento

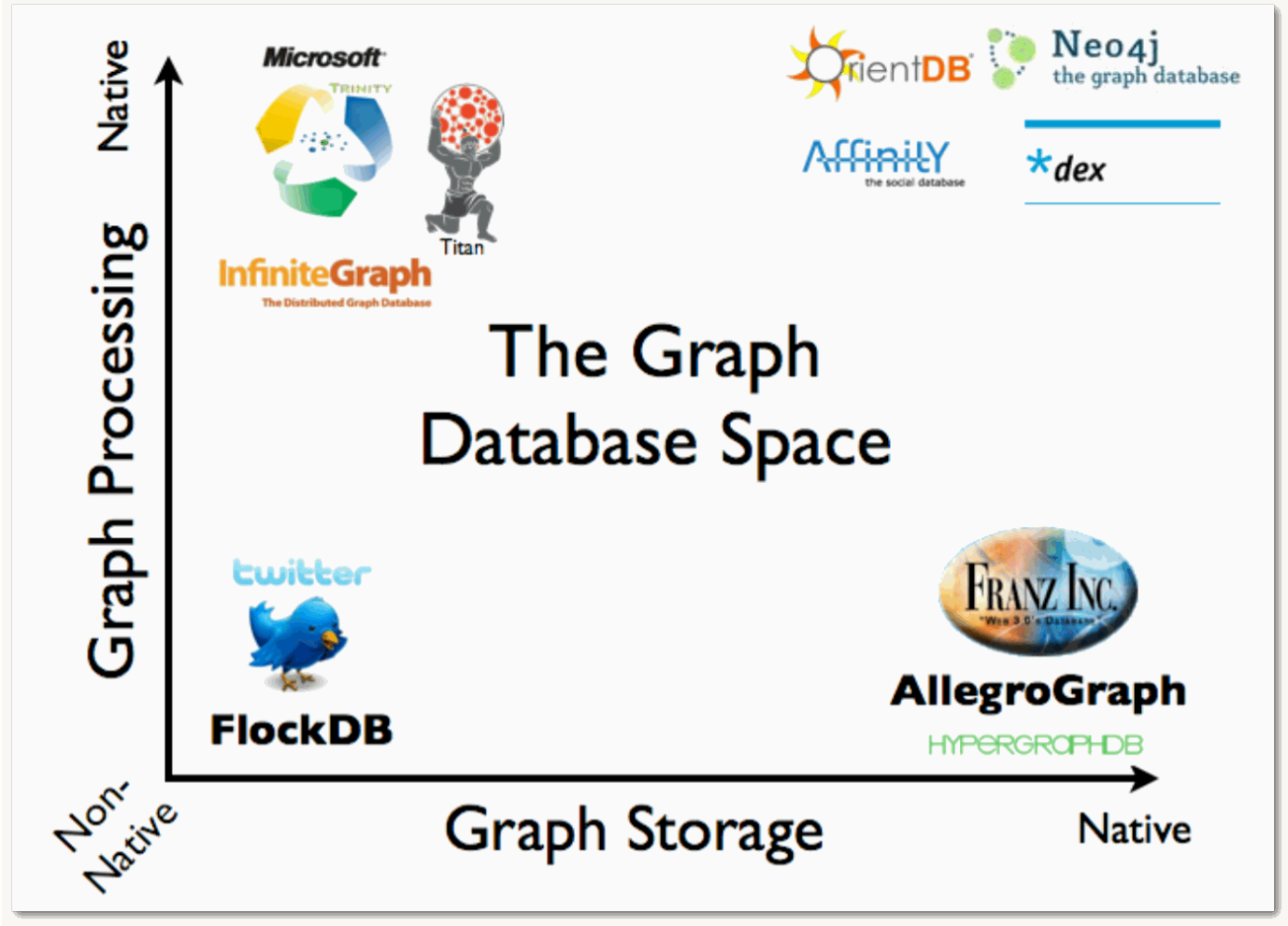
Neo4j es *Property Graph*

Tipos de BBDD de grafos

Se pueden clasificar atendiendo a dos dimensiones:

- **Almacenamiento subyacente**: nativo de grafos, o bien relacional/de otro tipo
- **Motor de procesamiento**: motor de grafos donde las entidades «apuntan» físicamente a otras (*index-free adjacency*), o bien se simula el procesamiento de grafos usando índices

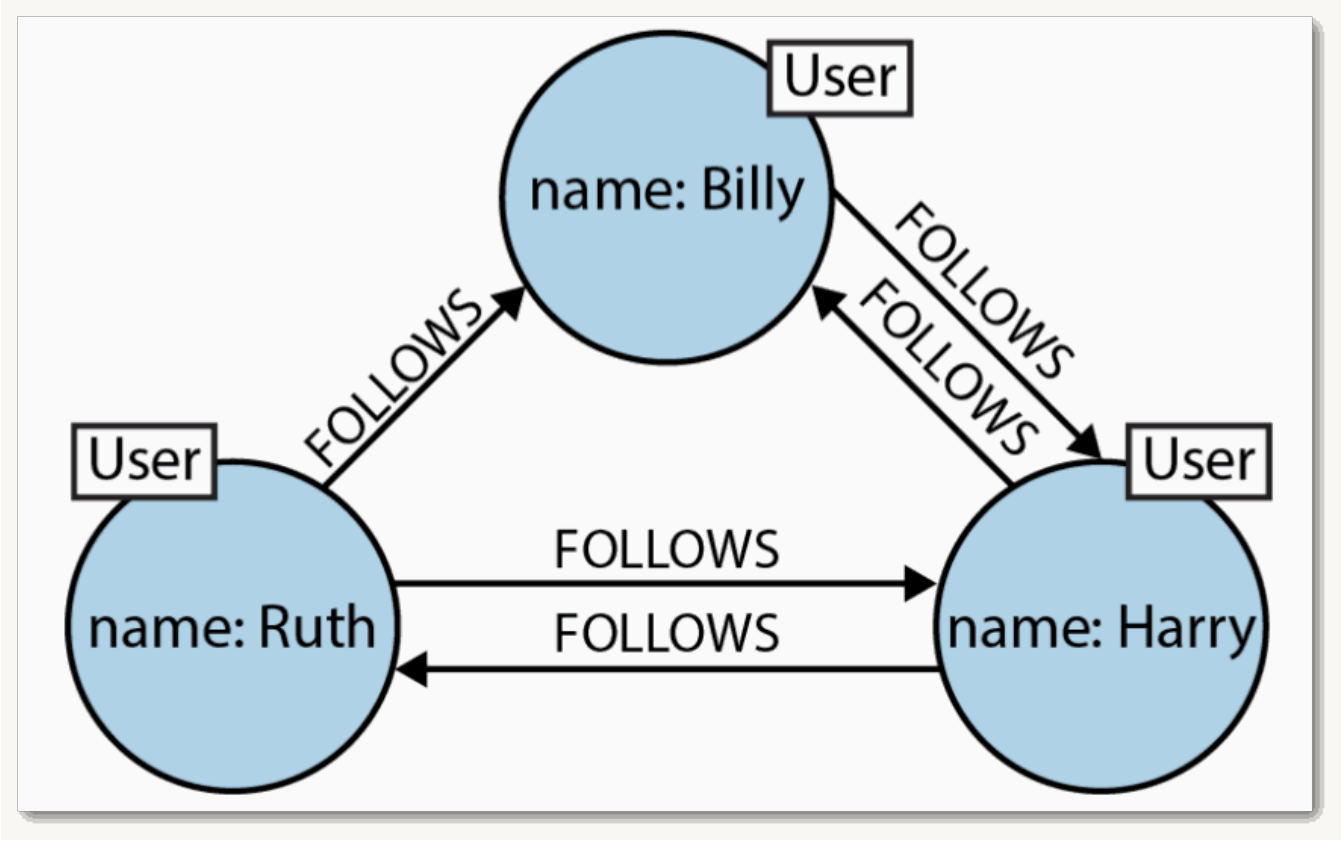
Ejemplos por cuadrante: nativo/nativo — **Neo4j**, InfiniteGraph; no nativo/nativo — OrientDB, JanusGraph; no nativo/no nativo — GraphDB sobre RDBMS


Native
Graph Processing
Non-Native
Graph Storage
Native
The Graph Database Space
Microsoft
TRINITY
Titan
InfiniteGraph
The Distributed Graph Database
OrientDB
Neo4j
the graph database
Affinity
the social database
*dex
twitter
FlockDB
FRANZ INC.
AllegroGraph
HYPERGRAPHDB

Aplicabilidad de grafos

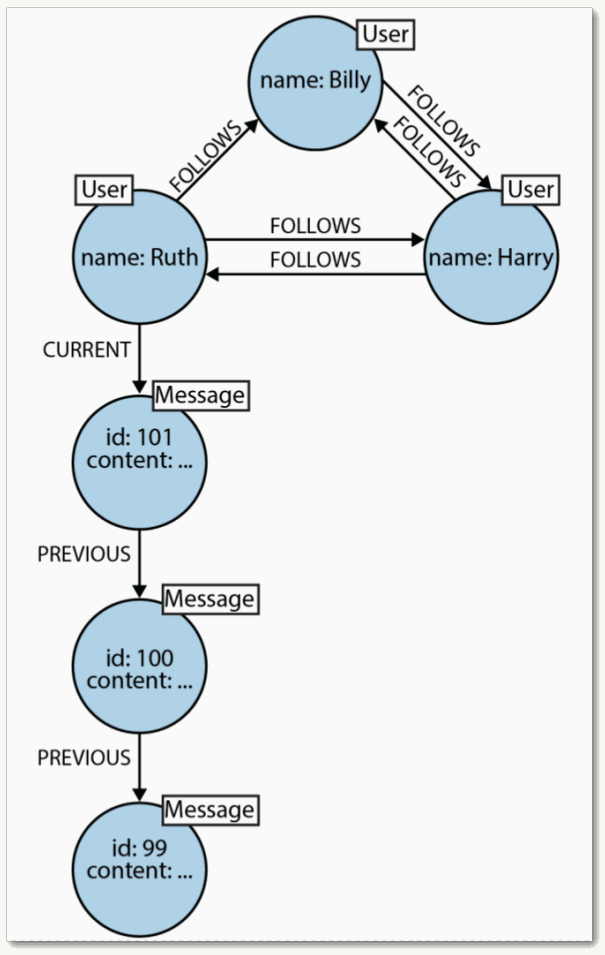
- Los grafos, conceptualmente, aparecen en casi cualquier dominio
- Su flexibilidad hace que se puedan aplicar de diferentes formas:
  - Gestión de identidad y acceso (IAM): permisos, roles, herencia
  - Cadenas de suministro: proveedores, productos, rutas
  - Análisis de dependencias de software: paquetes, versiones
  - Redes de transporte: estaciones, rutas, conexiones
  - Estructuras jerárquicas recursivas: organizaciones, taxonomías

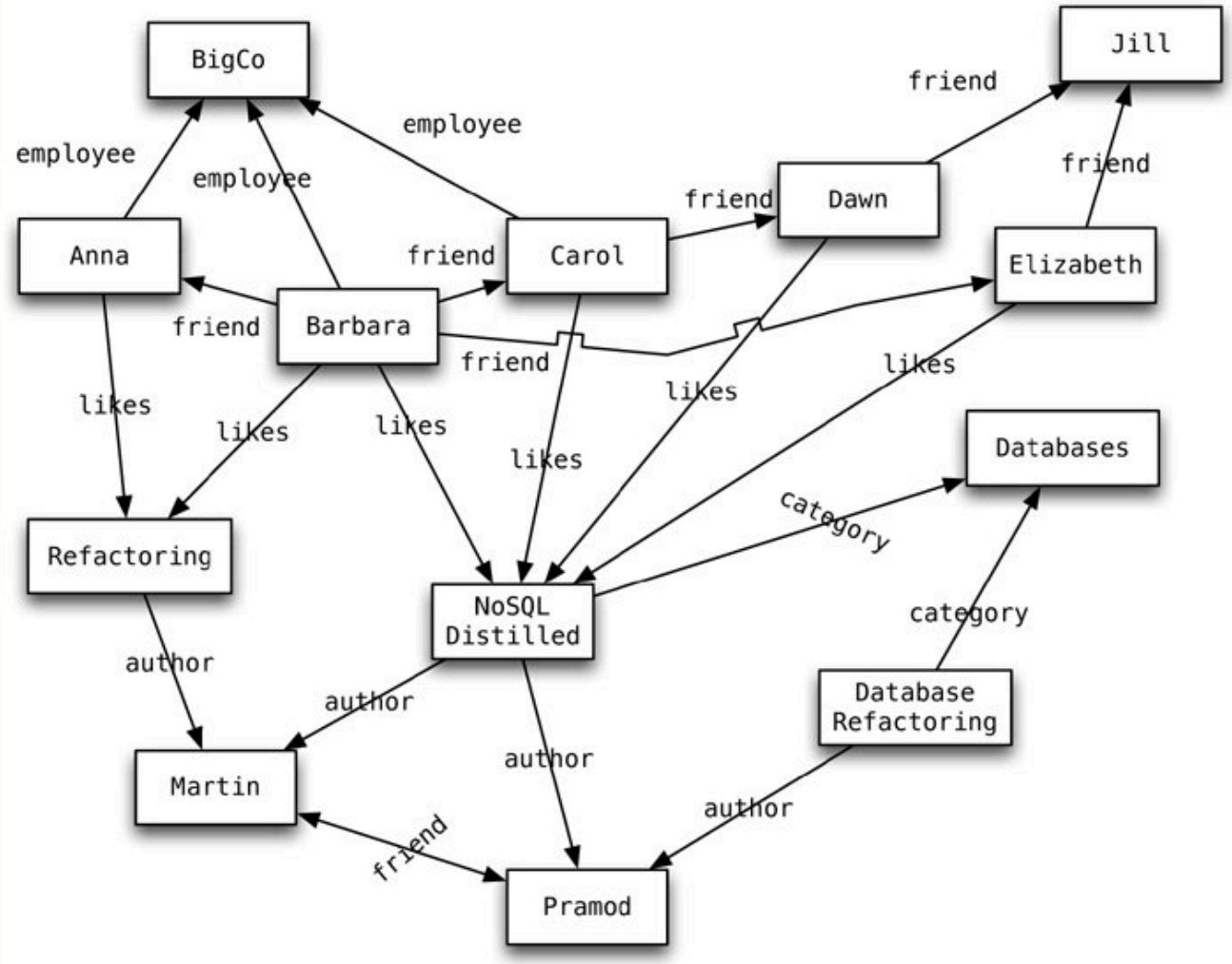
Ejemplo: relación *follow* entre usuarios



Nótese cómo **Billy no ha seguido a Ruth**: las relaciones pueden ser unidireccionales o bidireccionales

Guardando también los mensajes intercambiados




BigCo
Jill
employee
employee
employee
friend
friend
Dawn
Anna
friend
Carol
Elizabeth
friend
Barbara
friend
friend
likes
likes
likes
likes
likes
likes
Databases
Refactoring
category
NoSQL
Distilled
category
author
author
Database
Refactoring
author
Martin
author
friend
Pramod

Flexibilidad y eficiencia

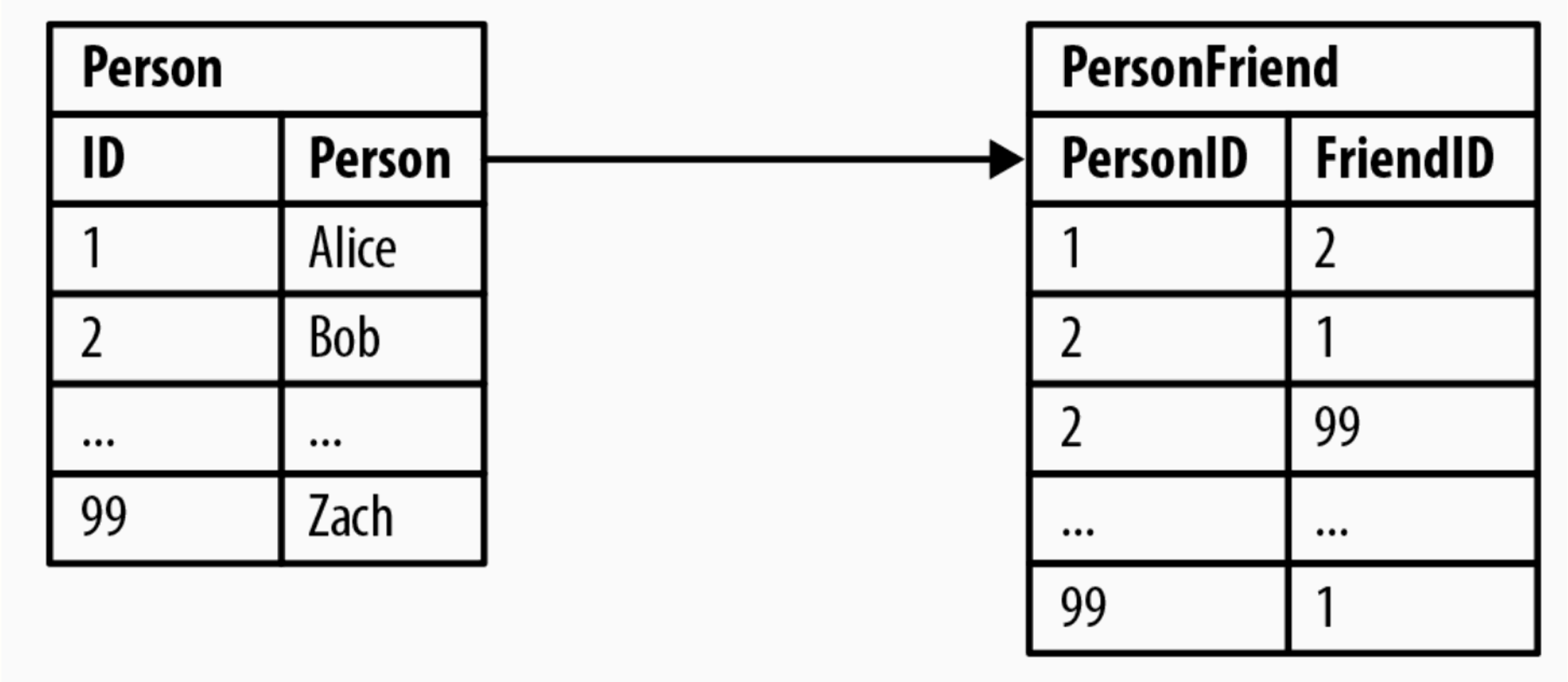
Las bases de datos de grafos suplen dos carencias fundamentales:

1. La carencia expresiva del resto de paradigmas para ciertas relaciones entre datos
2. La eficiencia del tratamiento de estos datos con algoritmos basados en grafos, frente a otros paradigmas

Clave de eficiencia: *index-free adjacency* — cada nodo mantiene referencias directas a nodos adyacentes, sin búsquedas en índices, tiempo constante O(1) para acceder a vecinos

> Ejemplo: una relación de amistad (*friend*) entre usuarios de una red social

Una posible implementación relacional



Person	
ID	Person
1	Alice
2	Bob
...	...
99	Zach

PersonFriend	
PersonID	FriendID
1	2
2	1
2	99
...	...
99	1

Consulta: amigos de Bob

```sql
SELECT p1.Person
FROM Person p1 JOIN PersonFriend
ON PersonFriend.FriendID = p1.ID
JOIN Person p2
ON PersonFriend.PersonID = p2.ID
WHERE p2.Person = 'Bob'
```

Es sencilla y no es computacionalmente muy compleja

Consulta inversa

Como la relación de amistad no siempre es recíproca, a veces hay que hacer la búsqueda inversa:

```
SELECT p1.Person
FROM Person p1 JOIN PersonFriend
ON PersonFriend.PersonID = p1.ID
JOIN Person p2
ON PersonFriend.FriendID = p2.ID
WHERE p2.Person = 'Bob'
```

Esta consulta es más costosa que la anterior, porque hay que recorrer todas las filas de `PersonFriend`

Amigos de los amigos de Alice

```sql
SELECT DISTINCT p1.Person AS PERSON, p2.Person AS FRIEND_OF_FRIEND
FROM PersonFriend pf1 JOIN Person p1
ON pf1.PersonID = p1.ID
JOIN PersonFriend pf2
ON pf2.PersonID = pf1.FriendID
JOIN Person p2
ON pf2.FriendID = p2.ID
WHERE p1.Person = 'Alice' AND pf2.FriendID <> p1.ID
```

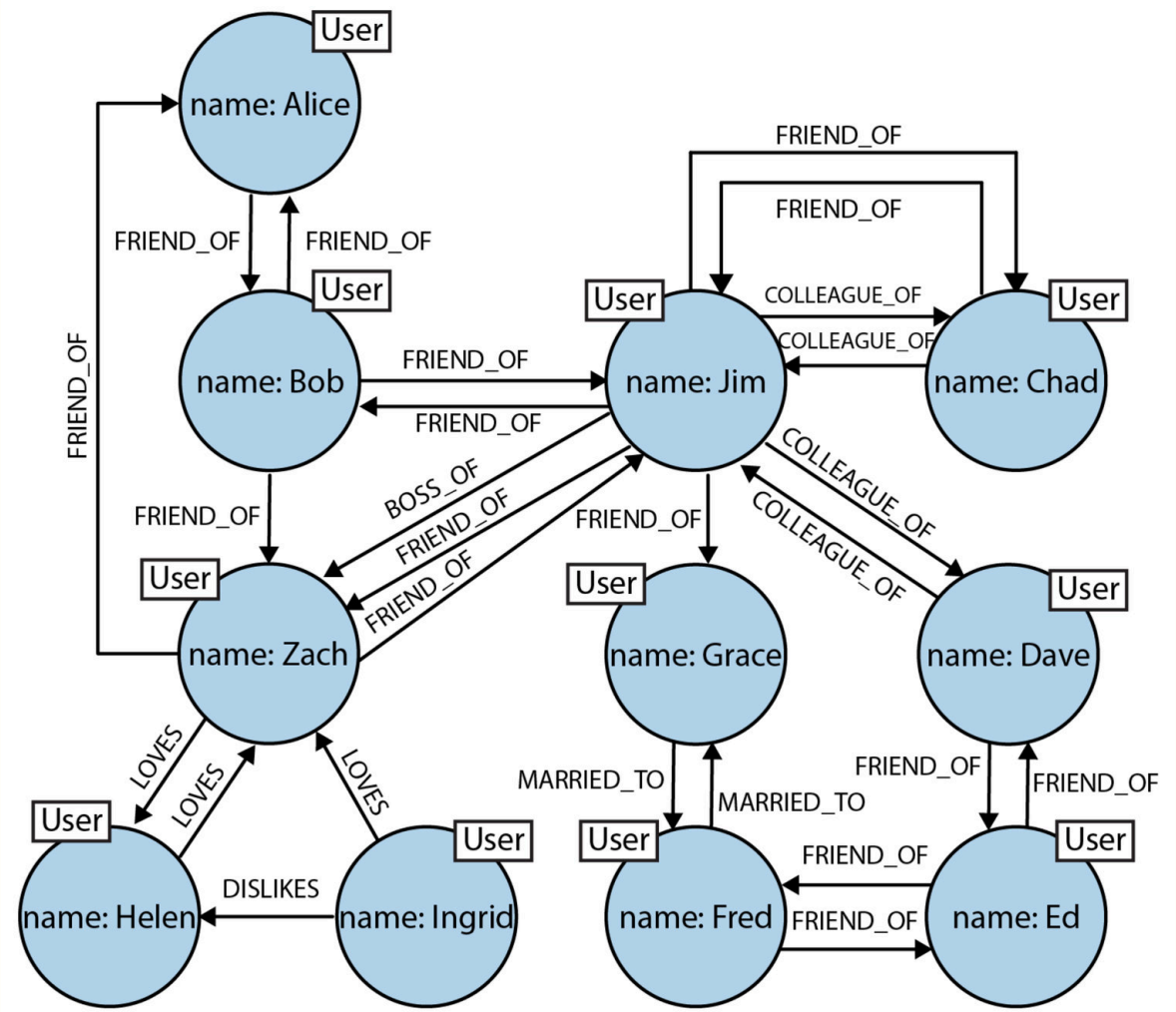
Y sólo estamos bajando **un** nivel. Si bajáramos otro, se complicaría mucho más por los `JOIN`

El experimento de *Neo4j in Action*

Partner y Vukotic compararon Neo4j y MySQL con esta tabla, buscando amigos de amigos en diferentes profundidades, en una red de 1 millón de personas con ~50 amigos cada una. (Experimento de 2014; el principio de eficiencia se mantiene en versiones modernas.)

Prof.	Tiempos RDBMS	Tiempos Neo4j	Resultados
2	0,016	0,01	~2 500
3	30,267	0,168	~110 000
4	1 543,505	1,359	~600 000

Nota: SQL puede ser mejor para operaciones masivas de agregación (OLAP) sobre todo el *dataset*


User
name: Alice
FRIEND_OF
FRIEND_OF
User
name: Bob
FRIEND_OF
FRIEND_OF
FRIEND_OF
FRIEND_OF
User
name: Jim
FRIEND_OF
COLLEAGUE_OF
COLLEAGUE_OF
User
name: Chad
BOSS_OF
FRIEND_OF
FRIEND_OF
FRIEND_OF
FRIEND_OF
COLLEAGUE_OF
COLLEAGUE_OF
User
name: Zach
User
name: Grace
User
name: Dave
LOVES
LOVES
LOVES
MARRIED_TO
MARRIED_TO
FRIEND_OF
FRIEND_OF
User
name: Helen
DISLIKES
User
name: Ingrid
User
name: Fred
FRIEND_OF
FRIEND_OF
User
name: Ed

Neo4j

Introducción a Neo4j

- Base de datos de grafos con posibilidad de extenderse a varios ordenadores (sólo uno soporta escritura, replicación *master-slave*)
- **Versiones**: Community (gratuita, una instancia, límites de autenticación), Enterprise (*clustering*, *backup* online, seguridad avanzada), Neo4j Aura (cloud gestionado, DBaaS)
- Compatible con el estándar Apache TinkerPop
- Lenguaje de creación y consulta de grafos: **Cypher**
- *Browser* web y consola para lanzar consultas; **Neo4j Desktop** para desarrollo local

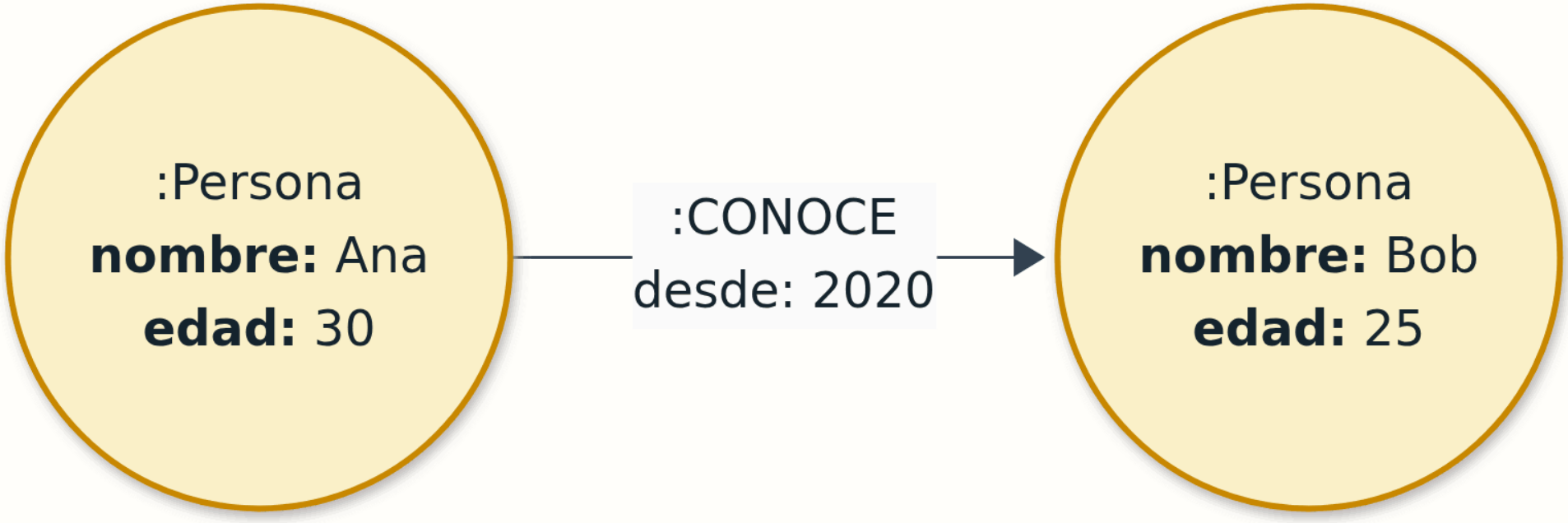
Introducción a Neo4j (ii)

- Multitud de algoritmos de recorrido de grafos, con la posibilidad de añadir más
- **APOC Library**: procedimientos y funciones avanzadas, muy utilizada
- **Graph Data Science (GDS)**: librería de algoritmos de grafos (PageRank, comunidades, etc.)
- Se puede usar desde Jupyter Notebook a través de una extensión
- **Drivers oficiales**: Python, Java, JavaScript, .NET, Go
- Ofrece también todos sus servicios a través de un API REST

Grafos en Neo4j

- Grafos etiquetados y con propiedades, compuestos por **nodos**, **relaciones**, **propiedades** y **etiquetas**
- Los nodos contienen propiedades como pares clave-valor: las claves son cadenas, los valores tipos primitivos o *arrays*
- **Tipos de datos soportados**: String, Integer, Float, Boolean, Point, Date, DateTime, LocalTime, Duration, y listas de primitivos
- A los nodos se les puede etiquetar con una o más ***etiquetas***, que los agrupan por rol dentro del grafo
- Las relaciones conectan nodos: **siempre tienen una dirección** (aunque se puede ignorar en consultas), un nombre propio, un nodo de inicio y otro de fin
- Como los nodos, las relaciones también tienen propiedades (p. ej. el peso o la calidad de un enlace), lo que además simplifica las consultas

Modelo de datos visual en Neo4j



Nodos (círculos) con etiquetas y propiedades; relaciones dirigidas con tipo y propiedades opcionales

Acceso desde Python

El *driver* oficial `neo4j` ofrece una API síncrona sobre el protocolo *Bolt*:

```
from neo4j import Driver, EagerResult, GraphDatabase, basic_auth

cypher_query: str = """
MATCH (n)
RETURN n AS id
LIMIT 10;
"""

driver: Driver = GraphDatabase.driver(BOLT_URL, auth=basic_auth("neo4j", ""))

result: EagerResult = driver.execute_query(cypher_query)
for record in result.records:
    print(record["id"])
```

`execute_query()` abre, ejecuta y cierra la transacción en un solo paso; para transacciones explícitas se usa `driver.session()`

El lenguaje Cypher

El lenguaje Cypher

Lenguaje de especificación de búsquedas y modificaciones en el grafo

Nodos:

```
( nombre:Etiqueta { propiedad: valor, ... } )
```

Relaciones, usando *ASCII art*:

```
(nodo_origen)-[:RELACIÓN]→(nodo_destino)
```

Cypher: relaciones

Con propiedades:

```
(n1)-[:RELACIÓN {año: 2017}]→(n2)
```

En el otro sentido:

```
(nodo1)←[:RELACIÓN]-(nodo2)
```

Omitiendo relaciones y nodos intermedios:

```
(nodo1)-[:RELACIÓN]→()-[:RELACIÓN2]→(nodo2)
(nodo1)⟶(nodo3)
```

Cypher: repetición de relaciones

Repetida un número acotado o no acotado de veces:

```
(nodo1)-[:RELACIÓN*1..10]→(nodo2)
(nodo3)-[:RELACIÓN*0..]→(nodoX)
```

Conjunto no limitado de relaciones, de forma general:

```
(nodo1)-[*]-(nodo2)
```

Cypher: `CREATE`

Creación de nodos y enlaces:

```
CREATE (nodo1:Etiqueta1 { propX: valorX, ... } ),
       (nodo2:Etiqueta2 { propY: valorY, ... } ),
        ...
       (nodo1)-[:RELACIONADO_CON]→(nodo2)
```

Consultas: `MATCH`

Consulta a través de ejemplos (*query by example*): especifica nodos y propiedades como ejemplo de lo que se busca

```
MATCH especificación, especificación, ...
[WHERE especificación]
[ORDER BY campo]
[LIMIT x]
RETURN [DISTINCT] nodos
```

`MATCH` también se usa para seleccionar nodos a crear (`CREATE`) o borrar (`DELETE` / `DETACH DELETE`). Retorna todos los *posts*:

```
MATCH (n:Post) RETURN n
```

Consultas: restricciones

```
MATCH (n { nombre: 'Diego' }) RETURN n.edad;
```

En este caso se devuelve una tabla, no un subgrafo

Consultas: nodos anónimos y caminos

Todos los abuelos con sus nietos:

```
MATCH (nieto), (abuelo),
   (nieto)-[:HIJO_DE]→()-[:HIJO_DE]→(abuelo)
 RETURN abuelo, nieto
```

O incluso (nótense los nodos anónimos):

```
MATCH (nieto)-[:HIJO_DE]→()-[:HIJO_DE]→(abuelo)
  RETURN abuelo, nieto
```

También:

```
MATCH (nieto)-[:HIJO_DE*2]→(abuelo)
  RETURN abuelo, nieto
```

Del patrón a las filas: Stack Overflow

En el grafo, `User` representa un usuario; `Question` y `Answer` son también nodos `Post`

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
      ←[:WROTE]-(autor_pregunta:User)
WHERE u.Id = 2
RETURN a.Id, q.Id, autor_pregunta.Id
```

- `WROTE`: autor → publicación; `ANSWERS`: respuesta → pregunta
- Buscamos las respuestas del usuario 2 y quién escribió cada pregunta
- Cada coincidencia del patrón produce una fila con `u`, `a`, `q` y `autor_pregunta`
- Este patrón exige que ambas publicaciones tengan autor representado en el grafo

Un ejemplo pequeño para seguir las filas

Datos **ilustrativos**, con los nombres y relaciones del boletín: Luis (`Id: 2`) escribió cuatro respuestas, dos de ellas a la misma pregunta

Respuesta `a.Id`	Pregunta `q.Id`	Autor `autor_pregunta.Id`
21	10	1 (Ana)
22	10	1 (Ana)
23	11	1 (Ana)
24	12	3 (Marta)

Hay **cuatro respuestas**, **tres preguntas** y **dos autores de preguntas**

> Repetir una pregunta en las filas no significa que haya nodos duplicados: hay varias coincidencias del patrón

`WITH`: pasar de una etapa a otra

`WITH` proyecta las filas intermedias: elige variables, calcula expresiones y les da nombres con `AS`

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
WHERE u.Id = 2
WITH a.Id AS respuesta, q.Id AS pregunta
RETURN respuesta, pregunta
ORDER BY respuesta
```

- Antes del `WITH`: cada fila contiene `u`, `a` y `q`
- Después: contiene sólo `respuesta` y `pregunta`; siguen siendo cuatro filas en el ejemplo
- `RETURN u, a` después de ese `WITH` daría error: ya no están en su ámbito
- `WITH *` conserva todas las variables; enumerarlas hace explícito qué necesita la siguiente etapa

Agregaciones: ¿qué estamos contando?

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
      ←[:WROTE]-(autor_pregunta:User)
WHERE u.Id = 2
RETURN count(a) AS respuestas,
       count(DISTINCT q) AS preguntas,
       count(DISTINCT autor_pregunta) AS autores
```

Resultado: **4 respuestas, 3 preguntas y 2 autores**

`count(a)` cuenta sus apariciones no nulas; `DISTINCT` cuenta valores diferentes de la expresión elegida

> Si ampliamos el patrón con las etiquetas de cada pregunta, una respuesta puede aparecer varias veces: para contar respuestas usaríamos `count(DISTINCT a)`

Agregación y agrupación implícita

Las expresiones proyectadas que no se agregan determinan los grupos

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
      ←[:WROTE]-(autor_pregunta:User)
WHERE u.Id = 2
RETURN autor_pregunta.Id AS autor, count(a) AS respuestas
ORDER BY autor
```

autor	respuestas
1	3
3	1

Sin `autor_pregunta.Id` se obtiene un único recuento global; al incluirlo, uno por autor

`WITH`: agregar y filtrar el resultado

¿A qué autores ha respondido Luis más de una vez?

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
      ←[:WROTE]-(autor_pregunta:User)
WHERE u.Id = 2
WITH autor_pregunta, count(a) AS respuestas
WHERE respuestas > 1
RETURN autor_pregunta.Id AS autor, respuestas
```

- El `MATCH` produce las cuatro filas del ejemplo
- `WITH` las agrupa por nodo autor: Ana → 3; Marta → 1
- El segundo `WHERE` deja sólo Ana → 3, como un `HAVING` en SQL
- `a`, `q` y `u` dejan de estar disponibles después del `WITH`

Consultas con `OPTIONAL MATCH`

Queremos contar respuestas **incluyendo las preguntas sin respuesta**, como con un `LEFT JOIN` en SQL

```
MATCH (q:Question)
OPTIONAL MATCH (q)<-[:ANSWERS]-(a:Answer)
RETURN q.Id AS pregunta, count(a) AS respuestas
ORDER BY pregunta
```

- `MATCH` establece las preguntas que queremos conservar
- Si hay respuestas, el patrón opcional produce una fila por coincidencia
- Si no hay ninguna, conserva la fila de `q` y asigna `null` a `a`
- `count(a)` ignora ese `null` y devuelve `0`; `count(*)` contaría la fila y daría 1

`OPTIONAL MATCH`: observar la ausencia

Añadimos al ejemplo la pregunta 13, sin respuestas

Filas antes de agregar: `q.Id`	`a.Id`
10	21
10	22
13	`null`

Para estas dos preguntas, `count(a)` da 2 y 0; `count(*)` da 2 y 1

- Con un `MATCH` obligatorio, la pregunta 13 desaparecería
- El `null` está en el resultado de la consulta: no hemos creado una respuesta ficticia ni una propiedad nula en el grafo

`OPTIONAL MATCH`: dónde se coloca el filtro

Conservar todas las preguntas y contar sólo sus respuestas con puntuación positiva:

```
MATCH (q:Question)
OPTIONAL MATCH (q)←[:ANSWERS]-(a:Answer)
WHERE a.Score > 0
RETURN q.Id, count(a) AS respuestas_positivas
```

- Este `WHERE` forma parte del patrón opcional
- Si ninguna respuesta cumple el filtro, se conserva la pregunta con `a = null` y el recuento es `0`
- También se conservan las preguntas que no tienen ninguna respuesta

Un filtro posterior puede eliminar las ausencias

Esta variante **no** conserva todas las preguntas:

```
MATCH (q:Question)
OPTIONAL MATCH (q)←[:ANSWERS]-(a:Answer)
WITH q, a
WHERE a.Score > 0
RETURN q.Id, count(a) AS respuestas_positivas
```

- `WITH` pasa las filas, incluidas las que tienen `a = null`
- El `WHERE` posterior sólo conserva las filas donde la condición es verdadera
- Se pierden las preguntas sin respuestas y las que sólo tienen respuestas sin puntuación positiva

> Para conservarlas, el filtro debe pertenecer al `OPTIONAL MATCH`, como en la transparencia anterior

Funciones de agregación

Contar publicaciones por identidad de usuario, sin mezclar homónimos:

```
// Contar posts por usuario
MATCH (u:User)-[:WROTE]→(p:Post)
RETURN u.Id AS usuario, u.DisplayName AS nombre, count(p) AS num_posts
ORDER BY num_posts DESC, usuario
```

`DisplayName` no es único: agrupar sólo por él sumaría las publicaciones de usuarios diferentes con el mismo nombre

Para incluir usuarios sin publicaciones, separar `MATCH (u:User)` y `OPTIONAL MATCH (u)-[:WROTE]→(p:Post)`

Funciones de agregación (ii)

```
// Promedio de edad de usuarios
MATCH (u:User)
RETURN avg(u.edad) AS edad_promedio
```

```
// Recopilar amigos en una lista
MATCH (p:Persona {nombre: 'Ana'})-[:AMIGO_DE]→(amigo)
RETURN p.nombre, collect(amigo.nombre) AS amigos
```

Otras funciones: `sum()`, `max()`, `min()`, `collect()`, `percentileDisc()`

Funciones de *strings*

```
MATCH (p:Post)
RETURN p.Title, toUpper(p.Title), toLower(p.Title)
```

```
MATCH (p:Post)
WHERE p.Title CONTAINS 'Neo4j'
RETURN p.Title
```

```
// split, substring, replace, trim
RETURN split("a,b,c", ",") AS lista
```

Funciones de listas

```
MATCH (u:User)-[:AMIGO_DE]→(amigo)
WITH u, collect(amigo) AS amigos
RETURN u.nombre, size(amigos) AS num_amigos,
       head(amigos) AS primer_amigo
```

Otras: `tail()`, `last()`, `range()`, `reduce()`

`UNWIND`: descomponer listas

`UNWIND` convierte una lista en filas individuales:

```
UNWIND [1, 2, 3] AS numero
RETURN numero
```

Útil para procesar listas de datos:

```
WITH ['Alice', 'Bob', 'Charlie'] AS nombres
UNWIND nombres AS nombre
CREATE (:Persona {nombre: nombre})
```

`UNWIND` con `collect`

```
MATCH (u:User)
WITH collect(u.email) AS emails
UNWIND emails AS email
RETURN toLower(email) AS email_normalizado
```

Variables de caminos

Se pueden asignar caminos completos a variables:

```
MATCH p = (a:Persona)-[:AMIGO_DE*1..3]→(b:Persona)
WHERE a.nombre = 'Ana' AND b.nombre = 'Carlos'
RETURN p, length(p)
```

Útil para algoritmos de caminos:

```
MATCH p = shortestPath(
  (a:Persona {nombre: 'Ana'})-[*]-(b:Persona {nombre: 'Carlos'})
)
RETURN p, length(p) AS distancia
```

Todos los caminos más cortos

```
MATCH p = allShortestPaths(
  (a:Persona {nombre: 'Ana'})-[*]-(b:Persona {nombre: 'Carlos'})
)
RETURN p
```

Filtros en relaciones

```
MATCH (u:User)-[r:AMIGO_DE WHERE r.desde > 2020]→(amigo)
RETURN u.nombre, amigo.nombre, r.desde
```

También en patrones de camino variable:

```
MATCH (a)-[r:CONOCE*1..3 WHERE r.confianza > 0.7]→(b)
RETURN a, b
```

`UNION`: combinar resultados

`UNION` elimina duplicados; `UNION ALL` los mantiene

```
// Usuarios activos: escribieron post O comentaron
MATCH (u:User)-[:WROTE]→(:Post)
RETURN u.nombre AS usuario
UNION
MATCH (u:User)-[:COMMENTED]→(:Comment)
RETURN u.nombre AS usuario
```

Las columnas devueltas deben coincidir en tipo y número

`UNION ALL`: mantiene duplicados

```
MATCH (u:User)-[:WROTE]→(:Post)
RETURN u.nombre AS usuario, 'post' AS actividad
UNION ALL
MATCH (u:User)-[:COMMENTED]→(:Comment)
RETURN u.nombre AS usuario, 'comment' AS actividad
```

`CALL`: procedimientos

- `CALL` ejecuta procedimientos almacenados
- Neo4j incluye muchos procedimientos *built-in*; APOC añade 450+ procedimientos más

```
// Listar todos los procedimientos
CALL dbms.procedures()
YIELD name, signature, description
WHERE name STARTS WITH 'apoc'
RETURN name, signature
LIMIT 10
```

`CALL`: con y sin resultados

```
// Ejecutar procedimiento con resultados
CALL db.labels() YIELD label
RETURN label, count(*) AS count
```

```
// Procedimiento sin resultados (standalone)
CALL db.clearQueryCaches()
```

`CALL`: procedimientos útiles

```
// Ver esquema del grafo
CALL db.schema.visualization()

// Estadísticas de nodos
CALL db.stats.retrieve('NODES')

// Crear constraint dinámicamente con APOC
CALL apoc.schema.assert(
  {User: ['email']},
  {User: ['id']}
)
```

Identidad y escrituras repetibles

Del identificador del dataset a una carga que podemos repetir y mantener

La identidad antes de la carga

`Id` identifica al usuario; su nombre visible puede cambiar o repetirse

```
CREATE CONSTRAINT user_id_unique IF NOT EXISTS
FOR (u:User) REQUIRE u.Id IS UNIQUE;
CREATE CONSTRAINT post_id_unique IF NOT EXISTS
FOR (p:Post) REQUIRE p.Id IS UNIQUE;
CREATE CONSTRAINT tag_name_unique IF NOT EXISTS
FOR (t:Tag) REQUIRE t.TagName IS UNIQUE;
```

- `Question` y `Answer` llevan también la etiqueta `Post`: comparten su restricción de identidad
- Cada restricción de unicidad respalda las búsquedas con un índice
- Unicidad no exige existencia: la carga debe comprobar que llega un identificador válido

`CREATE`: ¿qué ocurre al repetirlo?

Sobre el usuario 2 y el post 21 ya existentes en el ejemplo:

```
MATCH (u:User {Id: 2}), (p:Post {Id: 21})
CREATE (u)-[:WROTE]→(p)
```

- `MATCH` liga las variables a los nodos existentes; si falta uno, no hay fila que escriba
- `CREATE` crea una relación por fila, aunque ya exista otra `WROTE` entre esos nodos
- Ejecutarla dos veces añade dos relaciones: la unicidad de `User.Id` y `Post.Id` no lo impide
- Crear otra vez un nodo `User {Id: 2}` sí violaría la restricción y produciría un error

`MERGE`: buscar o crear el patrón

```
MATCH (u:User {Id: 2}), (p:Post {Id: 21})
MERGE (u)-[:WROTE]→(p)
```

- Con los extremos ya ligados, busca esa relación; si no existe, la crea
- Repetir la consulta no añade otra relación equivalente
- `MERGE` no limpia duplicados que se hayan creado anteriormente
- Si también hay que crear nodos, se resuelve primero su identidad y después la relación

> `CREATE` expresa «añade otro»; `MERGE` expresa «asegura que exista este patrón»

`MERGE` busca el patrón completo

Estado inicial: existen el usuario 2 y el post 21, pero no hay ninguna relación `WROTE` entre ellos

```
MERGE (u:User {Id: 2})-[:WROTE]→(p:Post {Id: 21})
```

- Busca el patrón entero: los dos nodos unidos por `WROTE`
- Como falta la relación, el patrón no coincide: intenta crear **dos nodos nuevos y la relación**, sin reutilizar por separado los existentes
- **Sin restricciones de unicidad:** crea otros nodos con los mismos `Id`; quedan duplicadas las entidades
- **Con las restricciones sobre `User.Id` y `Post.Id`:** falla y no deja creado ese nuevo patrón

> Si el patrón completo ya existiera, lo encontraría y no crearía nada

Resolver los nodos antes de la relación

Mismo estado inicial: existen el usuario 2 y el post 21; falta la relación `WROTE`

```
MERGE (u:User {Id: 2})
MERGE (p:Post {Id: 21})
MERGE (u)-[:WROTE]→(p)
```

1. El primer `MERGE` encuentra el usuario existente y lo liga a `u`
2. El segundo encuentra el post existente y lo liga a `p`
3. El tercero busca la relación entre **esos dos nodos ya determinados** y, como falta, crea sólo la relación

Estado final: los mismos dos nodos, unidos por una relación `WROTE`; repetir la consulta no añade nada

Si faltara algún nodo, su `MERGE` lo crearía antes de resolver la relación

Cambiar el nombre de un usuario

Estado inicial: existe `User {Id: 2, DisplayName: 'Luis'}`

Objetivo: cambiar su nombre a `'Luis García'`, conservando el mismo usuario

```
MERGE (u:User {Id: 2})
SET u.DisplayName = 'Luis García'
```

- `MERGE` lo reconoce por `Id: 2`, aunque todavía se llame `'Luis'`
- `SET` cambia el nombre del nodo que acabamos de localizar

Estado final: el mismo nodo, ahora `User {Id: 2, DisplayName: 'Luis García'}`, con sus relaciones intactas

Si el usuario 2 no existiera, `MERGE` lo crearía y `SET` le asignaría el nombre

Intento incorrecto de cambiar el nombre

Estado inicial: usuario 2 con nombre `'Luis'`, como antes de la actualización

```
MERGE (u:User {Id: 2, DisplayName: 'Luis García'})
```

- **No actualiza el nombre**: busca un nodo que ya tenga ambos valores
- `'Luis'` no coincide con `'Luis García'`: intenta crear otro nodo
- **Sin unicidad:** quedan dos usuarios con `Id: 2`, uno con cada nombre
- **Con unicidad sobre `User.Id`:** falla; el usuario sigue llamándose `'Luis'`

> `MERGE` reconoce la entidad; `SET` cambia sus atributos. Un nombre nuevo no debe convertir al mismo usuario en otro distinto

Distinguir el alta de la actualización

```
MERGE (u:User {Id: $id})
ON CREATE SET u.DisplayName = $nombre, u.Reputation = 0
ON MATCH SET u.DisplayName = $nombre
RETURN u.Id, u.DisplayName, u.Reputation
```

- Si se crea, recibe nombre y reputación inicial
- Si existe, se actualiza el nombre y se conserva su reputación
- `SET` sin estas condiciones se aplica tanto al crear como al encontrar
- Los valores de este ejemplo son ilustrativos; en la carga del dataset se usa su reputación real

`UNWIND` y `MERGE`: cargar un lote

El cliente envía `$filas`, una lista de mapas con `Id` y `DisplayName`, cuyos identificadores ha validado

```
UNWIND $filas AS fila
MERGE (u:User {Id: fila.Id})
SET u.DisplayName = fila.DisplayName
```

- `UNWIND` produce una fila Cypher por mapa; se envía una petición por lote
- `MERGE` resuelve la identidad y `SET` actualiza los atributos
- Repetir el mismo lote deja el mismo estado: la carga es **idempotente**
- Aquí `DisplayName` representa el valor completo de origen: si falta o es `null`, `SET` elimina esa propiedad; una actualización parcial debe distinguir «no cambiar» de «eliminar»

Materializar un recorrido frecuente

Definimos un interés como «el usuario ha preguntado sobre esa etiqueta», igual que en el boletín

```
MATCH (u:User)-[:WROTE]→(q:Question)-[:TAGGED_WITH]→(t:Tag)
WITH DISTINCT u, t
MERGE (u)-[:INTERESTED_IN]→(t)
```

- `WITH DISTINCT` deja una fila por pareja usuario–etiqueta antes de escribir
- La relación derivada permite consultar el interés sin repetir el recorrido por las preguntas
- Repetir esta consulta no añade otra `INTERESTED_IN` entre el mismo usuario y la misma etiqueta
- Si desaparece la última pregunta que justificaba ese interés, la relación queda guardada: la aplicación debe eliminarla, porque esta consulta no lo hace

Índices y restricciones

Localizar los nodos de partida y garantizar las reglas del modelo

Índices

Se pueden crear índices para búsquedas rápidas sobre atributos:

```
CREATE INDEX post_title_idx IF NOT EXISTS
FOR (n:Etiqueta) ON (n.nombre_campo)
```

Constraints: unicidad y existencia

Constraints de unicidad:

```
CREATE CONSTRAINT usuario_email_unique IF NOT EXISTS
FOR (u:Usuario) REQUIRE u.email IS UNIQUE;

CREATE CONSTRAINT post_id_unique IF NOT EXISTS
FOR (p:Post) REQUIRE p.Id IS UNIQUE;
```

Constraints de existencia (sólo en la edición Enterprise):

```
CREATE CONSTRAINT usuario_id_exists IF NOT EXISTS
FOR (u:Usuario) REQUIRE u.id IS NOT NULL;
```

Índices compuestos

Un índice sobre varias propiedades de la misma etiqueta:

```
CREATE INDEX usuario_nombre_apellido IF NOT EXISTS
FOR (u:Usuario) ON (u.nombre, u.apellido);
```

- Sirve a las consultas que filtran a la vez por `nombre` y `apellido`
- Existe también en la edición Community, la de las prácticas

Índices *full-text*

```
CREATE FULLTEXT INDEX postContent IF NOT EXISTS
FOR (n:Post) ON EACH [n.title, n.body];

// Uso
CALL db.index.fulltext.queryNodes(
  'postContent', 'Neo4j AND graphs'
) YIELD node, score
RETURN node.title, score
```

Ver índices y *constraints*:

```
SHOW INDEXES;
SHOW CONSTRAINTS;
```

Y consultas interesantes en Neo4j

```
CREATE
  (NAmerica:Location {name:'North America', type:'continent'}),
  (USA:Location {name:'United States', type:'country' }),
  (Idaho:Location {name:'Idaho', type:'state' }),
  (Lucy:Person {name:'Lucy' }),
  (Idaho)-[:WITHIN]->(USA)-[:WITHIN]->(NAmerica),
  (Lucy) -[:BORN_IN]->(Idaho)
```

```
MATCH
(person)-[:BORN_IN]->()-[:WITHIN*0..]->(us:Location {name:'United States'}),
(person)-[:LIVES_IN]->()-[:WITHIN*0..]->(eu:Location {name:'Europe'})
RETURN person.name
```

Modelado de datos en grafos

Principios de modelado en grafos

- **Empezar por las preguntas**, no por los datos: ¿qué consultas necesitamos responder? Diseñar el modelo para optimizarlas
- ***Whiteboard-friendly***: dibujar el modelo en una pizarra
- **Principio**: los datos conectados son tan importantes como los datos mismos
- **Desnormalizar**: a diferencia de RDBMS, duplicar datos puede ser beneficioso
- **Evolución**: el modelo puede evolucionar sin migraciones complejas

Nodos vs. propiedades

Usar nodos cuando:

- El elemento tiene identidad propia, hay que buscar por él o tiene relaciones con otros elementos
- Ejemplo: Usuario, Producto, Ciudad

Usar propiedades cuando:

- Son atributos simples del nodo, sin búsquedas frecuentes ni relaciones propias
- Ejemplo: nombre, edad, precio, fecha

Ejemplo: ciudad como nodo si se necesita «todas las personas en Madrid»; como propiedad si sólo se guarda el lugar de nacimiento sin más consultas

Relaciones vs. propiedades

Usar relaciones cuando: conectan dos entidades, la conexión tiene semántica importante, hay que navegarla, o puede tener propiedades (peso, fecha, tipo)

Usar propiedades cuando: el valor es un dato simple del nodo, sin necesidad de navegación

Ejemplo: «Trabaja en» como relación si se quiere «empleados de la empresa X»; como propiedad si sólo se guarda el nombre, sin navegación

Una relación puede tener propiedades

Si sólo necesitamos conectar actor y película y guardar el nombre del papel, basta esta representación:

```
(:Actor)-[:ACTÚA_EN {rol: 'Batman'}]→(:Película)
```

- Es un modelo válido: la relación puede guardar también año o caché
- Añadir propiedades no obliga a convertir la relación en un nodo
- Aquí el papel es un texto; no tiene identidad ni relaciones propias

> La decisión depende de qué queremos representar y consultar

Modelado de relaciones n-arias

Ahora queremos relacionar **actor, película y personaje**, cada uno con identidad propia

```
(actor:Actor)-[:REALIZA]→(actuación:Actuación)
(actuación)-[:EN_PELÍCULA]→(película:Película)
(actuación)-[:INTERPRETA]→(personaje:Personaje)
```

- Las relaciones de Neo4j conectan dos nodos; `Actuación` reúne las tres entidades mediante relaciones binarias
- Podemos conectar varias actuaciones al mismo personaje y buscar todas las interpretaciones de Batman en distintas películas
- Año y caché pueden ser propiedades de `Actuación`

> El nodo intermedio expresa una participación con identidad y conexiones propias

Modelado temporal: *Time-Tree Pattern*

El árbol permite localizar un año o mes y recorrer sus días hasta los eventos

```
(:Year)-[:HAS_MONTH]→(:Month)-[:HAS_DAY]→(:Day)
(:Evento)-[:OCURRIO_EN]→(:Day)
```

```
// Consulta: eventos en noviembre 2025
MATCH (:Year {valor: 2025})-[:HAS_MONTH]→(:Month {valor: 11})
      -[:HAS_DAY]→(d:Day)←[:OCURRIO_EN]-(evento:Evento)
RETURN evento
```

Se recorren **todos los días del mes**: no se filtra por un día concreto

Es una alternativa a guardar una propiedad temporal e indexarla; su utilidad depende de las consultas

Fan-out y *fan-in*

- **Fan-out**: un nodo con muchas relaciones salientes (p. ej. usuario con miles de posts) — puede ser lento al cargar el nodo
- **Fan-in**: un nodo con muchas relaciones entrantes (p. ej. *tag* popular en miles de posts) — problema de rendimiento similar
- **Solución**: usar nodos intermedios o diferentes tipos de relaciones
  - Agrupar por fecha, categoría, etc
  - Ejemplo: Usuario → Año → Mes → Posts

Algoritmos de grafos

Graph Data Science Library (GDS)

- Librería de Neo4j para algoritmos de grafos avanzados; se instala como *plugin* separado
- Tres categorías principales:
  i. **Pathfinding**: encontrar caminos y rutas óptimas
  ii. **Centralidad**: identificar nodos importantes
  iii. **Detección de comunidades**: encontrar grupos conectados
  iv. **Similitud**: encontrar nodos similares
- Operación típica: crear proyección del grafo en memoria, ejecutar el algoritmo, y escribir o analizar los resultados

Pathfinding: *shortest path*

Camino más corto entre dos nodos:

```
MATCH p = shortestPath(
  (a:Ciudad {nombre: 'Madrid'})-[*]-(b:Ciudad {nombre: 'Barcelona'})
)
RETURN p, length(p)
```

Pathfinding: Dijkstra

Camino más corto con pesos:

```
MATCH (a:Ciudad {nombre: 'Madrid'}), (b:Ciudad {nombre: 'Barcelona'})
CALL gds.shortestPath.dijkstra.stream({
  sourceNode: a, targetNode: b,
  relationshipWeightProperty: 'distancia'
})
YIELD path, totalCost
RETURN path, totalCost
```

- **A***: camino con heurística (más rápido)
- **All Shortest Paths**: todos los caminos más cortos

Algoritmos de centralidad

PageRank: importancia basada en conexiones — original de Google, identifica nodos influyentes

```
CALL gds.pageRank.stream({
  nodeProjection: 'User',
  relationshipProjection: 'FOLLOWS'
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).nombre AS usuario, score
ORDER BY score DESC LIMIT 10
```

- **Betweenness Centrality**: nodos que actúan como puentes
- **Closeness Centrality**: nodos más cerca del resto
- **Degree Centrality**: basado en número de conexiones

Algoritmos de comunidades

Louvain: detecta comunidades optimizando modularidad

```
CALL gds.louvain.stream({
  nodeProjection: 'User',
  relationshipProjection: 'FRIENDS_WITH'
})
YIELD nodeId, communityId
RETURN communityId, count(*) AS size
ORDER BY size DESC
```

- **Label Propagation**: propagación de etiquetas para comunidades
- **Weakly Connected Components**: componentes conectados
- **Triangle Count**: detecta *clustering*

Algoritmos de similitud

Node Similarity: nodos con vecinos similares

```
CALL gds.nodeSimilarity.stream({
  nodeProjection: 'User',
  relationshipProjection: 'LIKES'
})
YIELD node1, node2, similarity
WHERE similarity > 0.5
RETURN gds.util.asNode(node1).nombre,
       gds.util.asNode(node2).nombre,
       similarity
```

- **Jaccard Similarity**: basado en intersección de vecinos
- **Cosine Similarity**: para vectores de propiedades — útil para sistemas de recomendación

Rendimiento y buenas prácticas

Evitar productos cartesianos

```
// MAL - Cartesian product
MATCH (u:User), (p:Post)
WHERE u.id = p.userId
RETURN u, p

// BIEN - Usar relaciones
MATCH (u:User)-[:WROTE]→(p:Post)
RETURN u, p
```

Usar índices

```
// Crear índice primero
CREATE INDEX user_email IF NOT EXISTS
FOR (u:User) ON (u.email);

// Consulta aprovecha el índice
MATCH (u:User {email: 'user@example.com'})
RETURN u
```

Usar parámetros

Para reutilización de planes:

```
// Sin parámetros - nuevo plan cada vez
MATCH (u:User {email: 'user1@example.com'}) RETURN u
MATCH (u:User {email: 'user2@example.com'}) RETURN u

// Con parámetros - plan reutilizado
MATCH (u:User {email: $email}) RETURN u
```

Limitar temprano

¿Cuántas respuestas tienen las diez preguntas con mayor puntuación?

```
MATCH (q:Question)
WITH q ORDER BY q.Score DESC, q.Id LIMIT 10
OPTIONAL MATCH (q)<-[:ANSWERS]-(a:Answer)
RETURN q.Id AS pregunta, q.Score AS puntuación,
       count(a) AS respuestas
ORDER BY puntuación DESC, pregunta
```

- Seleccionamos por `Score` y desempatamos por `Id`; sólo después recorremos las respuestas
- `OPTIONAL MATCH` conserva las preguntas sin respuestas, con recuento 0
- Podemos limitar antes porque la selección no depende del recuento posterior

Analizar rendimiento: `EXPLAIN`

Muestra el plan de ejecución sin ejecutar:

```
EXPLAIN
MATCH (u:User {email: $email})-[:WROTE]→(p:Post)
RETURN u, p
```

Útil para ver si usa índices

Analizar rendimiento: `PROFILE`

Ejecuta y muestra estadísticas detalladas:

```
PROFILE
MATCH (u:User {email: $email})-[:WROTE]→(p:Post)
RETURN u, p
```

Muestra: filas procesadas en cada paso, *db hits* (accesos a disco/caché) y tiempo de ejecución

Buscar: alto *db hits*, muchas filas procesadas innecesariamente

Operaciones masivas: *batching*

Para operaciones masivas, usar transacciones por lotes; no insertar/actualizar millones de nodos en una sola transacción:

```
// Cargar CSV en lotes de 1000
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row
CALL {
  WITH row
  MERGE (u:User {id: row.id})
  SET u.nombre = row.nombre, u.email = row.email
} IN TRANSACTIONS OF 1000 ROWS
```

APOC para procesamiento paralelo

```
CALL apoc.periodic.iterate(
  "MATCH (u:User) RETURN u",
  "SET u.processed = true",
  {batchSize: 1000, parallel: true}
)
```

Configuración de memoria

- **Heap memory**: para transacciones y consultas. Típico 8–16 GB en producción; se configura con `dbms.memory.heap.max_size` en `neo4j.conf`
- **Page cache**: para cachear el grafo en memoria. Lo más grande posible (RAM disponible – *heap* – SO); se configura con `dbms.memory.pagecache.size`. Idealmente, todo el grafo en *page cache*
- **Regla general**: 50 % de la RAM para *page cache*, 25 % para *heap*
- **Monitorizar**: `:queries` en el *browser*, métricas JMX

Transacciones y concurrencia

- Neo4j es **ACID** completo
- Usa ***write locks*** en nodos/relaciones modificadas; los ***read locks*** no bloquean otras lecturas
- **Deadlocks** posibles en escrituras concurrentes: Neo4j los detecta y aborta la transacción automáticamente — hay que reintentarla
- **Buenas prácticas**: transacciones cortas, procesar en lotes (*batching*), usar `MERGE` con cuidado (puede causar conflictos), evitar modificar los mismos nodos desde múltiples *threads*

Comparación con SQL para Stack Overflow

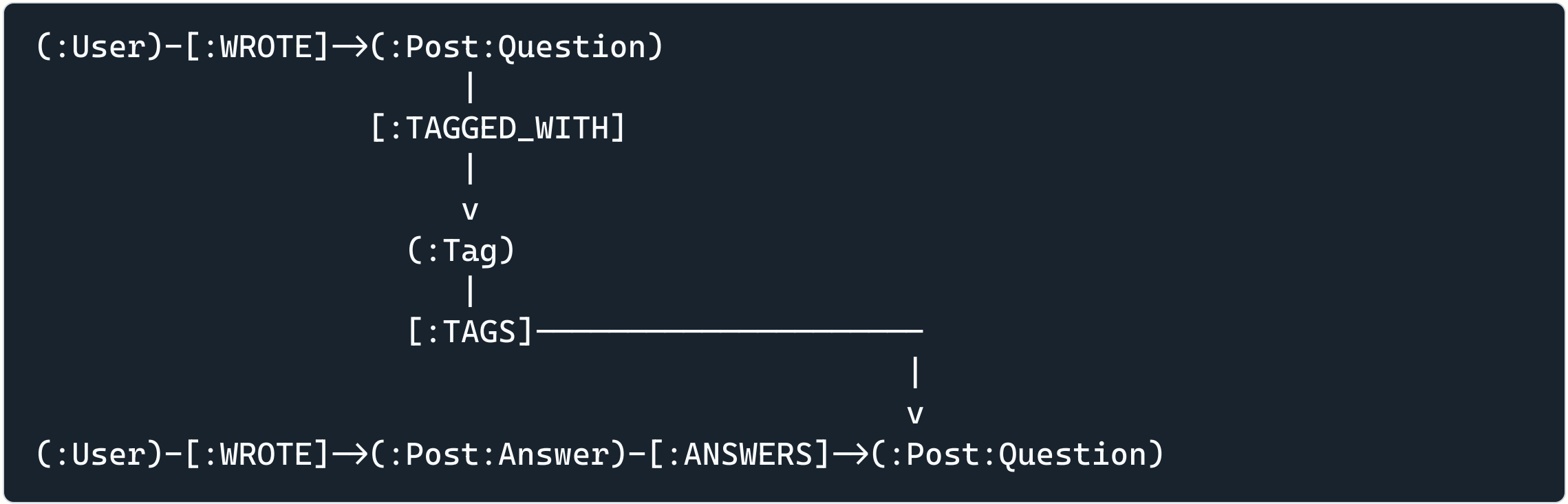
Modelo de datos de Stack Overflow

Entidades principales: `Users`, `Posts` (preguntas y respuestas), `Tags`, `Comments`

Relaciones:

- `User -[:WROTE]→ Post`
- `Post -[:ANSWERS]→ Post` (respuesta a pregunta)
- `Post -[:TAGGED_WITH]→ Tag`
- `User -[:COMMENTED]→ Comment -[:ON]→ Post`

Modelo visual de Stack Overflow



Dos relaciones para la misma asociación

El boletín guarda deliberadamente las dos direcciones de la asociación pregunta–etiqueta:

```
(q:Question)-[:TAGGED_WITH]→(t:Tag)
(t)-[:TAGS]→(q)
```

- Ambas relaciones conectan **la misma pregunta y la misma etiqueta**
- Una sola relación puede consultarse en ambos sentidos: `MATCH (t:Tag)←[:TAGGED_WITH]-(q:Question)` parte de la etiqueta en el patrón escrito
- Guardar ambas es redundante: al añadir o quitar una asociación hay que mantener las dos coherentes

> La dirección de una relación no impide consultarla desde su nodo de destino

Stack Overflow, SQL

La consulta de reciprocidad, sesión 2: parejas de usuarios que se han respondido mutuamente

```
CREATE VIEW Pairs AS SELECT DISTINCT Q.OwnerUserId AS questioner, A.OwnerUserId AS answerer
FROM Posts Q JOIN Posts A ON Q.AcceptedAnswerId = A.Id
AND Q.OwnerUserId <> A.OwnerUserId;

SELECT P1.questioner AS U1, P2.questioner AS U2
FROM Pairs P1, Pairs P2
WHERE P1.questioner = P2.answerer
  AND P2.questioner = P1.answerer
  AND P1.questioner < P2.questioner;
```

Stack Overflow, SQL (con CTE)

```sql
WITH Pairs AS (
  SELECT DISTINCT Q.OwnerUserId AS questioner, A.OwnerUserId AS answerer
  FROM Posts Q JOIN Posts A ON Q.AcceptedAnswerId = A.Id
  AND Q.OwnerUserId <> A.OwnerUserId
)
SELECT P1.questioner AS U1, P2.questioner AS U2
FROM Pairs P1, Pairs P2
WHERE P1.questioner = P2.answerer
  AND P2.questioner = P1.answerer
  AND P1.questioner < P2.questioner;
```

Stack Overflow, Neo4j

Consulta: parejas recíprocas

```
MATCH
    (u:User)-[:WROTE]→()-[:ANSWERS]→()←[:WROTE]-(u2),
    (u2:User)-[:WROTE]→()-[:ANSWERS]→()←[:WROTE]-(u)
WHERE toInt(u.Id) < toInt(u2.Id)
RETURN DISTINCT u.Id,u2.Id;
```

Crear relación `RECIPROCATE`

```
MATCH
    (u:User)-[:WROTE]→()-[:ANSWERS]→()←[:WROTE]-(u2),
    (u2:User)-[:WROTE]→()-[:ANSWERS]→()←[:WROTE]-(u)
WHERE toInt(u.Id) < toInt(u2.Id)
MERGE (u2)-[:RECIPROCATE]→(u)
MERGE (u)-[:RECIPROCATE]→(u2);
```

Top usuarios por respuestas aceptadas

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
WHERE q.AcceptedAnswerId = a.Id
RETURN u.DisplayName, count(a) AS respuestas_aceptadas
ORDER BY respuestas_aceptadas DESC
LIMIT 10
```

Preguntas sin respuesta

```
MATCH (q:Question)
WHERE NOT (q)<-[:ANSWERS]-()
RETURN q.Title, q.CreationDate
ORDER BY q.ViewCount DESC
LIMIT 100
```

Tags más relacionados (co-ocurrencia)

```
MATCH (t1:Tag)←[:TAGGED_WITH]-(q:Question)-[:TAGGED_WITH]→(t2:Tag)
WHERE t1.TagName < t2.TagName
RETURN t1.TagName, t2.TagName, count(q) AS veces
ORDER BY veces DESC
LIMIT 20
```

Expertos por *tag*

Más respuestas aceptadas en un *tag*:

```
MATCH (u:User)-[:WROTE]→(a:Answer)-[:ANSWERS]→(q:Question)
      -[:TAGGED_WITH]→(t:Tag {TagName: 'neo4j'})
WHERE q.AcceptedAnswerId = a.Id
RETURN u.Id AS usuario, u.DisplayName AS nombre,
       count(a) AS respuestas
ORDER BY respuestas DESC, usuario
LIMIT 10
```

Agrupamos también por `Id` para no sumar las respuestas de usuarios distintos con el mismo nombre visible

Sistema de recomendación simple

Recomendar amigos: amigos de mis amigos que no conozco

```
MATCH (yo:User {nombre: 'Ana'})-[:AMIGO_DE]→()-[:AMIGO_DE]→(recomendado)
WHERE NOT (yo)-[:AMIGO_DE]→(recomendado) AND recomendado <> yo
RETURN recomendado.nombre, count(*) AS amigos_comunes
ORDER BY amigos_comunes DESC
LIMIT 10
```

Recomendar productos

Basado en compras similares:

```
MATCH (yo:User {id: 123})-[:COMPRO]→(p:Producto)
      ←[:COMPRO]-(otro:User)-[:COMPRO]→(recomendacion:Producto)
WHERE NOT (yo)-[:COMPRO]→(recomendacion)
RETURN recomendacion.nombre, count(*) AS popularidad
ORDER BY popularidad DESC
LIMIT 10
```

Recomendar contenido

Por similitud de intereses:

```
MATCH (yo:User {id: 123})-[:INTERESTED_IN]→(t:Tag)
      ←[:INTERESTED_IN]-(similar:User)-[:WROTE]→(post:Post)
WHERE NOT (yo)-[:WROTE]→(post)
WITH post, count(DISTINCT similar) AS usuarios_similares
RETURN post.titulo, usuarios_similares
ORDER BY usuarios_similares DESC
LIMIT 10
```

Carga de datos desde CSV

- Neo4j puede cargar datos directamente desde archivos CSV
- Los archivos deben estar en `import/`, dentro del directorio de Neo4j

Cargar usuarios:

```
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row
CALL {
  WITH row
  MERGE (u:User {Id: row.Id})
  SET u.DisplayName = row.DisplayName,
      u.Reputation = toInteger(row.Reputation),
      u.CreationDate = datetime(row.CreationDate)
} IN TRANSACTIONS OF 1000 ROWS
```

Cargar *posts* y relaciones

```
LOAD CSV WITH HEADERS FROM 'file:///posts.csv' AS row
CALL {
  WITH row
  MERGE (p:Post {Id: row.Id})
  SET p.Title = row.Title,
      p.Body = row.Body,
      p.PostTypeId = toInteger(row.PostTypeId)

  // Crear relación con usuario
  WITH p, row
  MATCH (u:User {Id: row.OwnerUserId})
  MERGE (u)-[:WROTE]→(p)

  // Si es respuesta, crear relación ANSWERS
  WITH p, row
  WHERE row.ParentId IS NOT NULL
  MATCH (q:Post {Id: row.ParentId})
  MERGE (p)-[:ANSWERS]→(q)
} IN TRANSACTIONS OF 500 ROWS
```

Importación masiva

Para grandes volúmenes, `neo4j-admin import`:

```
neo4j-admin import \
  --nodes=User=users.csv \
  --nodes=Post=posts.csv \
  --relationships=WROTE=wrote.csv \
  --relationships=ANSWERS=answers.csv
```

Mucho más rápido (millones de nodos por minuto). Requiere base de datos vacía

Recursos y herramientas

Recursos de aprendizaje

- **GraphAcademy** (graphacademy.neo4j.com): cursos gratuitos online interactivos y certificaciones oficiales, de básico a avanzado
- **Neo4j Sandbox** (sandbox.neo4j.com): entornos precargados para experimentar (Movie Graph, Twitter, Game of Thrones, etc.), sin instalación
- **Documentación oficial** (neo4j.com/docs)
- **Comunidad**: Neo4j Community Forum, Stack Overflow (*tag* `neo4j`)
- **Canal de YouTube** de Neo4j: tutoriales y casos de uso

Herramientas y extensiones

- **Neo4j Desktop**: entorno de desarrollo local, gestión de varias bases de datos, *plugins* (APOC, GDS, Bloom)
- **Neo4j Bloom**: visualización y exploración de grafos, interfaz para usuarios no técnicos, búsqueda en lenguaje natural
- **APOC** (*Awesome Procedures On Cypher*): 450+ procedimientos y funciones; importación/exportación de datos; utilidades de texto, fecha, JSON
- **Neoclipse**: IDE Eclipse para Neo4j (descontinuado, pero útil)